



Structured query language injection attack detection via giant magnificent frigatebird optimization and federated learning

Madhavi Perla¹ · Valli Kumari Vatsavayi¹

Received: 16 April 2025 / Accepted: 24 November 2025 / Published online: 11 January 2026
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

Abstract

Structured Query Language injection (SQLi) attack detection is considered one of the most critical vulnerabilities in web applications, where an attacker injects malicious Structured Query Language (SQL) commands in the input fields. This causes an inconvenience to users and disrupts application functionality. To reduce errors and ensure the continued integrity and performance of web applications, a new method for SQLi attack detection is developed in this research. Here, Federated Learning (FL) is utilized to detect SQLi attacks to enable collaborative, decentralized model training across numerous devices without the requirement to share sensitive data. Initially, the submitted SQL queries are transformed into binary sequences. Next, one-hot encoding techniques, like Mutual Information, Entropy, Correlation coefficient, Information Gain, and Gini index, are applied to transform the query to a one-hot vector. Then, the processed data is fed to SQLi attack detection using Temporal Convolutional Long Short-Term Memory with Giant Magnificent Frigatebird Optimization (TempCLSTM_Gi-MFO). Moreover, TempCLSTM is trained by devising Giant Magnificent Frigatebird Optimization (Gi-MFO), which integrates Magnificent Frigatebird Optimization (MFO) and Giant Trevally Optimizer (GTO). Therefore, the TempCLSTM_Gi-MFO achieved an accuracy of 91.847%, F1-score of 91.680%, recall of 92.485%, and precision of 90.889% with a time step of 25 s.

Keywords Federated learning · Structured query language query · One-hot encoding · Structured query language injection attack detection · Deep learning

✉ Madhavi Perla
madhaviperla87@gmail.com

¹ Department of Computer Science and Systems Engineering, AU College of Engineering, Andhra University, Visakhapatnam 530003, India

1 Introduction

Federated Learning (FL) is a distributed Machine Learning (ML) framework and was devised in language prediction models on smartphones, effectively reaching the aim of preserving user privacy by preventing the leakage of personal data. FL serves as an efficient solution to tackle challenges related to data isolation and privacy concerns in ML (Jianping et al. 2024). FL is a recent advancement in Artificial Intelligence (AI) that operates on the principle of decentralized data. It has gained significant attention from the cybersecurity community as a method for collaboratively training Deep Learning (DL) approaches that leverage distributed profiles of cyber threats while ensuring that training data remains private. Software security is a critical quality metric for any software product, with SQL injection (SQLi) being one of the most serious security vulnerabilities that software development teams must address (Arasteh et al. 2024a). SQLi is a prevalent and complex method of network attacks, recognized as one of the top 10 web application security risks by the Open Web Application Security Project (OWASP) (Zhang et al. 2022). SQLi is a significant vulnerability in web applications that occurs when user-provided input is improperly handled in SQL queries, allowing attackers to manipulate the database and execute malicious commands (Thalji et al. 2023). SQLi is a critical weakness in web security, offering attackers the opportunity to interact with software applications. By exploiting SQL queries, attackers can access sensitive data from databases, potentially obtaining unlimited information and gaining unauthorized access to the system (Sefati et al. 2024). Most traditional security techniques are insufficient to effectively protect web application databases from SQLi attacks. These methods often fail to address the complex nature of SQLi, leaving systems vulnerable to exploitation (Arasteh et al. 2024b).

SQLi is a type of code injection attack where input data is treated as executable code, violating the fundamental principle of separating data from code. An attacker can inject malicious SQL commands into a web form submission, page request, thereby altering the execution logic of the SQL statement. This allows them to access restricted resources or modify the data, which is deposited in the dataset, especially when the web application passes the SQL statements to the backend file without filtering the input data of the user in a proper manner. Due to their ease of execution and high level of threat, SQLi attacks have become a popular tactic among cybercriminals (Lu et al. 2023). However, most existing solutions can only detect a limited subset of SQLi attacks and struggle to keep up with evolving attack methods. Given the wide range of SQLi attack types and their numerous variants, it is crucial to develop an effective DL-based detection system that can automatically perform feature extraction and adapt to new, sophisticated attack strategies (Zhang et al. 2022; Chen et al. 1757). SQLi attacks have gained popularity among cyber attackers due to their simplicity. Existing rule-based detection schemes are often inefficient and fail to detect different attacks. Additionally, the availability of numerous free SQLi tools online has made it easier for attackers to carry out these attacks (Crespo-Martínez et al. 2023). The faults in development languages, gaps in developers' expertise, and no awareness about web security significantly increase the likelihood of successful attacks on websites. Consequently, it is essential to develop a reliable and accurate SQLi attack detection model to enhance web application security (Sun et al. 2023).

In recent years, researchers have proposed various methods for detecting SQLi attacks, which can be broadly divided into three schemes such as traditional-based methods, ML-based methods, and Deep Neural Network (DNN)-based methods. Each scheme offers different advantages in terms of detection accuracy, as well as adaptability to evolve attack strategies. The traditional approach works well for string matching in simpler scenarios, but as attackers employ increasingly sophisticated tools for automated injection attacks, this method struggles to keep up. The traditional approach for addressing SQLi attack patterns has evolved, making it less effective in addressing modern threats. Traditional SQLi detection methodologies, including static and dynamic analysis, as well as instruction randomization, primarily rely on rule-based detection (Sun et al. 2023). The ML-based approach addresses some of the limitations of traditional methods by improving adaptability and detection accuracy. In addition, ML-based models often require manual feature extraction, which is time-consuming and may not capture all relevant attack patterns effectively (Zhang et al. 2022). DNN-based approaches eliminate the need for manual extraction of features when compared to traditional ML methods. ML algorithms have been successfully applied to create SQLi detectors, where classifiers are trained to distinguish between attack queries and normal SQL queries, improving the accuracy and efficiency of detection systems (Arasteh et al. 2024b; Deriba et al. 2022). Researchers have developed various techniques, including signature-based detection and anomaly-based detection, to enhance the accuracy of SQLi attack detection (Khan et al. 2017; Alarfaj and Khan 2023). However, these approaches often face challenges, such as low accuracy and an inability to effectively manage the growing complexity of modern SQLi attacks. As a result, developing a more effective strategy for detecting SQLi attacks remains a significant challenge and a critical priority. Currently, feature extraction is a key component of most SQLi detection methods. Researchers often focus on identifying explicit features within these attack statements and apply simple ML methods to detect the attacks. However, this approach may not capture the full complexity of SQLi attacks, limiting its effectiveness (Sun et al. 2023). As DL continues to advance, researchers have increasingly turned to it for detecting SQLi attacks due to its ability to capture complex patterns in data (Sun et al. 2023).

1.1 Research gap and motivation

SQLi attack is a serious threat to web applications and databases for protecting sensitive data, ensuring application security, and maintaining the integrity of web-based systems and databases. Despite several approaches being developed, they often cause inconvenience to users, increase processing time and system load, and potentially affect application performance and scalability. The traditional model failed to generalize different environments of SQL, applications, and users. Many models rely on basic tokenization features and fail to capture the semantic complexities of SQL queries. The existing methods ignored temporal patterns, which limit the ability to understand query flow and detect time-based or chained attacks. Also, some of the methods failed to handle long-term dependencies in the complex SQL queries. To address these limitations, a TempCLSTM_Gi-MFO is established to detect SQLi attacks, aiming to achieve improved performance and more efficient protection.

1.2 Contribution

This research proposes a new scheme of TempCLSTM_GiMFO for SQLi attack detection in FL. Initially, local training is performed using data available at each node. Subsequently, the weights are updated, and the model is then aggregated at a central server. Then, the training process is updated iteratively at each epoch based on the downloaded global model and the local model. In the local training model, the SQLi attack model is considered. Initially, the submitted SQL queries are transformed into binary sequences. Afterwards, one-hot encoding techniques, like Mutual Information, Entropy, Information Gain, Correlation coefficient, and Gini index, are applied to convert the query into a one-hot vector. Then, the processed data is passed to a hybrid DL model, named TempCLSTM, which is the combination of Temporal convolutional Networks (TCN) and Long Short-Term Memory (LSTM) for detecting SQLi attacks. Besides, TempCLSTM is trained using the established Gi-MFO, which is the fusion of MFO and GTO. Finally, the SQLi attack detector verifies SQL queries as normal SQL or hacker SQL.

The key contribution of this research is,

- *Devised TempCLSTM_Gi-MFO for SQLi attack detection:* The TempCLSTM system is developed for detecting SQLi attacks. TempCLSTM is devised by combining TCN and LSTM. Here, TempCLSTM is trained by using the devised Gi-MFO, which is the integration of MFO and GTO.

1.3 Structural organization

The residual section is ordered as follows: a comprehensive analysis of prior approaches for SQLi attack detection is provided in Sect. 2. The SQLi attack technique is deliberated in Sect. 3, and the entire workflow of the TempCLSTM_Gi-MFO method is expounded in Sect. 4. The results of the designed TempCLSTM_Gi-MFO method are exemplified in Sect. 5, and finally, research findings as well as future scope are summarized in Sect. 6.

2 Literature survey

The prevailing schemes related to SQLi attack detection are expounded below.

Thalji et al. (2023) developed an Autoencoder network (AE-Net) for detecting SQLi attacks. AE-Net handled large amounts of input data efficiently and was applied to different types of SQLi. Besides, AE-Net required fewer computational resources. However, the framework was not revised to minimize computational complexity and failed to progress a user-friendly graphical interface adapted for organizations, like networking and enable real-time monitoring of SQLi attacks. Alarfaj and Khan (2023) introduced a Probabilistic Neural Network (PNN) to detect SQLi attacks. PNN with the Bat algorithm achieved low false positive rates, attained high accuracy, and fast performance. But, PNN with BAT recorded high complexity, and had high sensitivity to noise on inappropriate features. Zhang et al. (2022) designed a framework

for SQLi detection using Neural Networks (SQLNN). SQLNN effectively managed false positives and preserved data integrity and centralized security. But, SQLNN was prone to overfitting if it was not tuned properly, and had limited flexibility and scalability. A.G. Kakisis designed the Bidirectional Long Short-Term Memory and Convolutional Neural Network (LSTM-CNN) based on Multi-View Consensus (MVC-BiCNN) for SQLi detection in Kakisis (2024). MVC-BiCNN improved detection performance and attained high robustness. However, MVC-BiCNN fails to address two key features, such as adapting the system to handle several forms of code injection attacks, like Cross-Site Scripting (XSS), and exploring system augmentation through dynamic monitoring approaches, which could offer additional promising solutions.

Lu et al. (2023) designed the syntactic Bidirectional Encoder Representations from Transformer (synBERT) for detecting SQLi attacks. The synBERT attained a high accuracy rate, minimized False Positives, and handled complex SQLi techniques. However, synBERT did not learn enough SQL dialects because of limited training data. Moreover, synBERT attained a high memory footprint due to its complex architecture. Arasteh et al. (2024b) designed an improved binary version of the Olympiad Optimization Algorithm (BOOA). BOOA enhanced accuracy, reduced overfitting, and increased the efficiency of the detection system by avoiding unnecessary computations on less impactful features. However, BOOA failed to examine how cloud computing could utilize various learning techniques, DL, and boosting algorithms to detect SQLi attacks effectively. Arasteh et al. (2024a) introduced the binary Gray Wolf Optimizer (bGWO) for detecting SQLi attacks that attained high accuracy and was capable of finding global optima or near-optimal solutions in complex, high-dimensional spaces. However, bGWO was computationally intensive and designed to handle binary problems, which may restrict its flexibility in certain SQLi detection scenarios. Sun et al. (2023) developed Text Convolutional Neural Networks (TextCNN) and an LSTM for SQLi detection. TextCNN and LSTM improved accuracy and reduced the false negative as well as false positive rates. In addition, the timeliness of discrimination demonstrates strong performance, resulting in excellent overall detection outcomes. However, TextCNN and LSTM failed to assess the robustness of the developed model to improve the stability.

2.1 Challenges

The difficulties faced by prior techniques for detecting SQLi attacks are detailed as follows,

- Thalji et al. (2023) AE-Net was applied to different kinds of SQLi for processing SQL queries to excerpt the features, which captured key patterns and features associated with SQLi attacks. However, AE-Net did not analyze the binary representations of queries for detecting malicious patterns effectively.
- Alarfaj and Khan (2023) PNN showed exceptional performance while detecting SQLi attacks. However, the PNN algorithm did not assist in mining and representing the appropriate characteristics from SQL queries that were important to detect anomalies efficiently.

- Zhang et al. (2022) the SQLNN model combined Rectified Linear Unit (ReLU) activation with dropout schemes, and a well-designed multi-hidden-layer framework for significantly enhancing the system performance, and making it a robust solution for detecting SQLi. However, the SQLNN scheme did not enhance the performance on complex temporal tasks that involve both local as well as global temporal dependencies.
- Kakisim (2024) MVC-BiCNN altered the SQL input as three different views by enhancing the feature space with several feature subgroups to improve the model's ability to capture diverse patterns and better detect SQLi attacks. However, MVC-BiCNN did not update the process of assessing the importance of features or selecting features depending on their contribution to minimize uncertainty.
- Detection system creates false positives, and flags genuine queries as malicious, which can lead to redundant alerts and minimize the efficiency of security measures. On the other hand, these systems may fail to detect certain attacks, mainly if the attack vectors are cleverly concealed.

3 SQL injection attack model

SQL (Alghawazi et al. 2023) is a programming language designed for managing and manipulating relational databases. SQL permits users to interact with a dataset by adding new data, removing outdated information, and adjusting previous records. However, SQLi attacks present a significant security risk to web applications. These attacks include the malicious implementation of SQL queries on a server, allowing illegal access to sensitive data and the potential retrieval of limited information stored inside the databases. The fundamental process involved in SQLi attack is illustrated in Fig. 1. SQLi attack model comprises a Web Application login form, Web Application Firewall (WAF), Web Server, and MySQL. Attackers exploit web applications

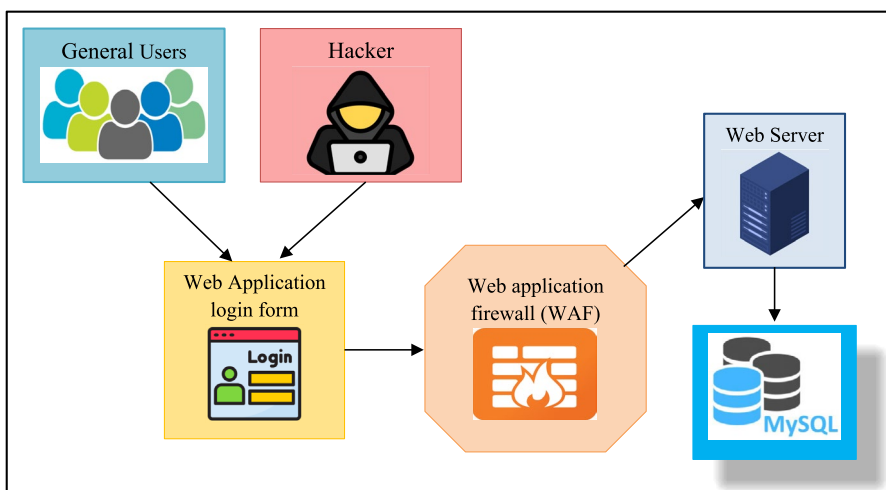


Fig. 1 SQLi attack model

by inserting SQL statements through user input to target the database layer, thereby gaining unauthorized admission to valuable data. This vulnerability often arises from a lack of proper input validation, which is due to programming errors. As a result, attackers avoid the authentication principle and gain access to databases by allowing them to retrieve or alter the data without proper permission.

4 Designed temporal convolutional long short-term memory with giant magnificent frigatebird optimization scheme for SQL injection attack model

This article introduces the TempCLSTM_Gi-MFO model for detecting SQLi attacks in an FL environment. Local nodes are the devices that train models with their local data in the FL setup. The local dataset is typically specific to the individual node and contains data about SQLi attacks from that particular environment. The local model computes the necessary gradients or updates after training and sends the model updates to the global node without sharing raw data. The TempCLSTM_Gi-MFO model is used in the local model for detecting SQLi attacks. The global node refers to a central server or aggregator that coordinates the FL process by collecting model updates from all local nodes. Then, it aggregates these updates to improve the global model. In this approach, each node performs local training using its own data to detect SQLi attacks, focusing on identifying malicious SQL queries. After the local training, the model updates are transmitted to a central server, where they are combined to create a global model, and then distributed to the nodes. Then, each node achieves local training based on the updated global model. This process is iteratively repeated at each epoch with the model that continuously improves its ability to detect SQL injection attacks through combined learning across all nodes. The fundamental framework of TempCLSTM_Gi-MFO for SQLi attack detection in FL is depicted in Fig. 2.

4.1 Load device training model

FL is regarded as a decentralized system that decreases communication overhead by carrying out training locally on the nodes while keeping the data stored on the devices themselves. The local data is exploited to adjust the training model for detecting the tasks after it is pre-processed. Besides, the performance of the local model is improved through leveraging the learned data locally. The primary objective of FL is to train models locally with available SQL queries. Once the local models are trained, their generated weights are shared with other nodes for building the global models.

4.1.1 Training at every node

The primary goal of each node is to optimize the local parameters with the smallest loss function during the phase of local training. The local node initiates by processing its own dataset, which consists of SQL queries and their labels. This step involves feature extraction or data normalization to prepare the data for training. After achiev-

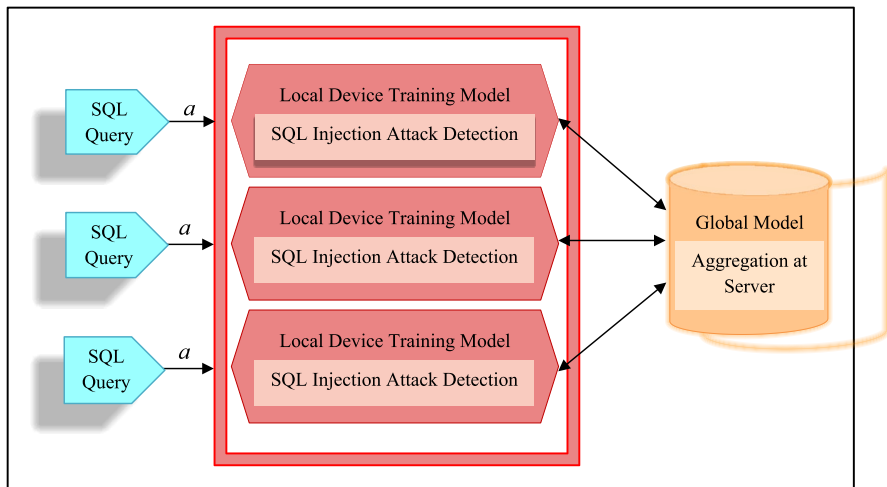


Fig. 2 The fundamental framework of TempCLSTM_Gi-MFO for SQLi attack detection in FL

ing this, the resulting parameters are passed to the server, where the weights from every node are aggregated to form a global model, and returned to the local nodes. This process is repeated until the designated number of epochs is completed. At each timestep a , the local nodes are trained to detect SQL queries with the training data sourced from the SQL Injection Dataset.

4.1.2 Training model of SQLi attack detection

In the training model of SQLi attack detection, the SQL queries are initially transformed into binary sequences. Binary sequences are used to represent the data using a series of binary digits. In the case of SQLi detection, SQL queries are often represented as binary sequences where each character, keyword, or operator in the query is converted into a binary value, such as 0 s and 1 s. Subsequently, one-hot encoding techniques, like, Mutual Information, Entropy-Based One-Hot Encoding, Correlation coefficient, Information Gain, and Gini index, are applied to convert the query into a one-hot vector. Furthermore, the processed data is then fed to the hybrid DL model, named TempCLSTM_Gi-MFO, to detect whether SQL is normal or a hacker. Here, TempCLSTM is the combination of TCN (Lea et al. 2017) and LSTM (Sagheer and Kotb 2019) that is used for detecting SQLi attacks. Moreover, TempCLSTM is trained using the devised Gi-MFO, which is the integration of MFO (Hamadneh et al. 2024a) and GTO (Sadeeq and Abdulazeez 2022). The objective model is then processed based on required resources, local and global loss function, and timing delay of the local and Global models. Further, local updation and aggregation at the server are changed depending on the average, and the aggregated model parameters in the cloud. Figure 3 indicates the training model of SQLi attack detection using the TempCLSTM_Gi-MFO approach.

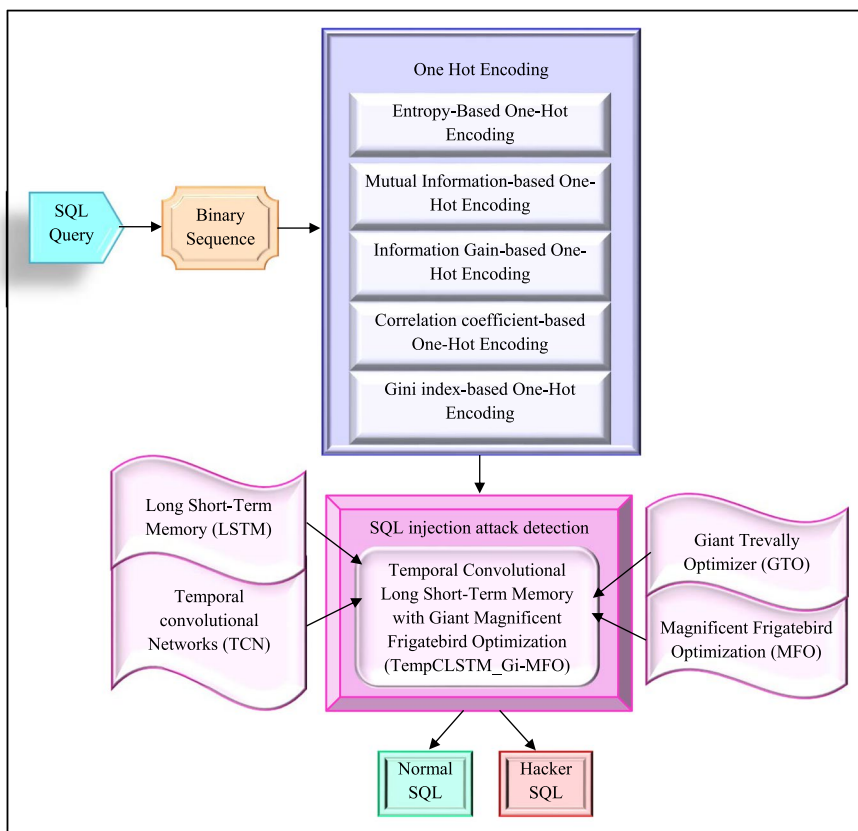


Fig. 3 The training model of SQLi attack detection using TempCLSTM_Gi-MFO

4.1.2.1 SQL query acquisition SQLi attack dataset (Y) (The SQL Injection Dataset) with $\mathfrak{R}$ number of SQL queries is considered for detecting SQLi attacks. The s^{th} input SQL query (Y_s) in the database (Y) is attained and forwarded to the following phase of binary sequence to detect SQLi attack.

$$Y = \{Y_1, Y_2, \dots, Y_s, \dots, Y_{\mathfrak{R}}\} \quad (1)$$

4.1.2.2 Binary sequence of SQL query and one hot encoding

A binary sequence of an SQL query is defined as the representation of a query in a binary format that uses the values (0 s and 1 s). During the processing of SQL queries, a binary sequence is obtained through several methods like tokenization, and one-hot encoding. The phases of converting a SQL query into a binary sequence. It helps to reduce noise, improve feature quality, and focus on the high-impact patterns. This ensures that only the most relevant tokens are represented in the one-hot vector.

It also helps to identify the most relevant and informative SQL tokens for detecting SQLi attacks. The one-hot encoding with an example is given below.

(i) Consider an SQL query

An SQL query is a command used for interacting with databases for defining, manipulating, and retrieving data. Consider an example SQL query for this process:

For instance: Select * from users where id=1 or “[” or 1=1__1.

(ii) Tokenize the SQL query

The process of breaking the query into smaller, meaningful components called tokens is referred to as Tokenization. The individual tokens, while considering the above queries, are indicated after tokenization as,

“Select” “*” “from” “users” “where” “id” “=” “1” “or” “[” “]” “” “or” “1” “=” “1” “_” “_” “1”

(iii) Allocate an index to each unique token

After tokenization, a unique index is assigned to each different token in the query, which essentially involves creating a mapping where each token is mapped to a unique index. Table 1 depicts the token indexing.

(iv) Convert each token to a one-hot vector where only the position corresponding to that token is set as one.

Table 1 Token indexing

Token	Index
Select	0
*	1
from	2
users	3
where	4
id	5
=	6
1	7
or	8
“	9
[	10
]	11
”	9
or	8
1	7
=	6
1	7
_ _	12
1	7

The following steps are followed to convert each token in an SQL query into a one-hot vector.

Step (i)—*Tokenize the SQL query*: Initially, the SQL query is broken into individual tokens.

Step (ii)—*Create a vocabulary*: A list of unique tokens is built into the query.

Step (iii)—*Create one-hot vectors*: For each token, generate a vector where the position corresponding to that token in the vocabulary is set to 1 and all other positions are set to 0.

Table 2 illustrates the One-hot vector conversion. Here, the token “Select” corresponds to index 0, so its one-hot vector is [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]. Similarly, the token “*” corresponds to index 1, so its one-hot vector is [0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]. This process continues for each token in the query. Each token in the SQL query is now represented as a one-hot vector. This encoding is used for various ML tasks, like training models to understand and manipulate SQL queries.

One-hot encoder is a sequence of vectors for the entire SQL query, with each vector represented as a token. The final depiction is the long binary vector found by concatenating every individual one-hot vector.

(v) Entropy-based one-hot encoding

Entropy (O_1) (Elayaraja and Suganthi 2018) is a measure of the uncertainty or unpredictability of information content. In the framework of one-hot encoding, entropy measures the amount of uncertainty in the distribution of the tokens and is formulated as,

$$O_1(C) = - \sum_{d=1}^v b(c_d) [\log_2 b(c_d)] \quad (2)$$

where c_d indicates the specific token, the probability of a specific token (d) in the set is indicated as $b(c_d)$, and the number of unique tokens is signified as v .

In the case of one-hot encoding, every token is encoded as a vector where only one position has a value of 1, and the remaining positions possess the value zero. The entropy is maximized when each token has an equal likelihood of occurring.

(vi) Mutual information based one-hot encoding

Mutual Information (O_2) (Hossain and Islam 2024) is a measure of the amount of information shared between two variables. In one-hot encoding, mutual information measures how much knowing the value of one token helps us predict the value of another token and is stated as,

$$O_2(C; D) = O_1(C) + O_1(D) - O_1(C, D) \quad (3)$$

Table 2 One-hot vector conversion

Token	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Select	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
*	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
from	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
users	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
where	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
id	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
=	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
or	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
“	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
[	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
]	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
”	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
or	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
=	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
-	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0

where, C and D signifies the tokens in SQL query, $O_1(C)$ and $O_1(D)$ implies the entropy of the token C and D . The joint entropy of C and D is explicated as $O_1(C, D)$.

Mutual information helps to identify the relationships between tokens when one-hot encoded tokens are applied.

(vii) Information gain based on one-hot encoding

Information Gain (O_3) (Prasetyo et al. 2021) measures the amount of information a token provides regarding the class. Information gain is defined as the reduction in entropy achieved by knowing a specific token.

$$O_3(C) = O_1(D) - O_1(D|C) \quad (4)$$

where the entropy of the target variable D is indicated as $O_1(D)$, and the conditional entropy of the target variable at a given feature C is exemplified as $O_1(D|C)$.

In one-hot encoding, the information gain of the token C is evaluated as the reduction in uncertainty regarding the target when knowing whether the token is present or not.

(viii) Correlation coefficient-based one-hot encoding

The correlation coefficient (O_4) (Benesty et al. 2008) is commonly known as Pearson correlation, which measures the linear relationship between two variables. For one-hot encoding, the correlation coefficient is computed between binary (0 or 1) vectors of different tokens.

$$O_4(C, D) = \frac{\sum_{f=1}^t (c_{C,f} - \bar{c}_C) (c_{D,f} - \bar{c}_D)}{\sqrt{\sum_{f=1}^t (c_{C,f} - \bar{c}_C)^2 \sum_{f=1}^t (c_{D,f} - \bar{c}_D)^2}} \quad (5)$$

where tokens are implied as C and D , context is explicated as f , co-occurrence count of token C in the context f is exemplified as $c_{C,f}$, mean of the co-occurrence counts of token C is designated as $\bar{c}_C$, and the number of co-occurrences is indicated as t .

(ix) Gini index-based one-hot encoding

Gini index (O_5) (Sadiqâ and Musawi 2018) is used to measure the impurity or purity of a database to assess how effectively a token divides the data. A lower Gini index indicates that the token is more effective at classifying the data into pure subsets, which means that the data points in each subset are more homogeneous with respect to the target variable.

$$O_5 = 1 - \sum_{C=1}^g b_C^2 \quad (6)$$

where the probability of the token C is represented b_C , and the number of unique tokens is indicated as g .

Further, one-hot encodings based on Mutual Information, Information gain, Correlation coefficient, and Gini index are combined and deliberated as $E = \{O_2, O_3, O_4, O_5\}$, which is subject to the SQLi attack detection phase.

4.1.2.3 SQL injection attack detection SQLi attack detection is the process of identifying and preventing malicious SQL queries from being executed on a database through user input or other input channels in a web application. The main intention is to identify suspicious behaviour, such as attempts to manipulate SQL queries, and prevent these attacks before they succeed in compromising the system's security. Here, an SQLi attack is detected using the TempCLSTM, which is the combination of TCN (Lea et al. 2017) and LSTM (Sagheer and Kotb 2019). Generally, TCNs allow for greater parallelization during training, and reduce the vanishing gradient problem, whereas LSTM permits for deeper framework and handles multiple types of sequential data. Therefore, TempCLSTM achieved better performance and improved handling of long-term dependencies. Furthermore, TempCLSTM is trained using the introduced GiMFO, which is the combination of MFO and GTO to enhance the convergence speed, and helps the model to handle complex, noisy, and imbalanced data.

(i) Architecture of TempCLSTM

The architecture of TempCLSTM is depicted in Fig. 4. TempCLSTM is the combination of TCN (Lea et al. 2017) and LSTM (Sagheer and Kotb 2019) and it encompasses three elements such as TCN, TempCLSTM layer, and LSTM. Initially, the entropy-based one-hot encoding is given as input to the TCN and creates the output L_1 . Furthermore, the one-hot encodings based on Mutual Information, Information gain, Correlation coefficient, and Gini index combined as E are subjected to the TempCLSTM layer along with the TCN output L_1 to generate the outcome L_2 . Then, the consequences of the TempCLSTM layer are fed to the LSTM for generating the outcome L_3 .

TCN: TCNs (Lea et al. 2017) perform layer-wise computation, which means that all time steps are updated simultaneously during the forward pass. The entropy-based one-hot encoding matrix feature (O_1) is considered as an input to TCN. TCNs perform convolutions across time and spatial dimensions. This enables the network to learn temporal dependencies in the data, which is vital for SQLi attack detection. Predictions at each time step depend on a fixed-length time, which is known as the receptive field. Encoder-Decoder TCN (ED-TCN) is designed to capture both contextual information and temporal features at different levels. In Temporal Convolutions, the encoder processes the input sequence, and the decoder generates the predictions at each time step based on learned temporal patterns. Rectified Linear Unit (ReLU)

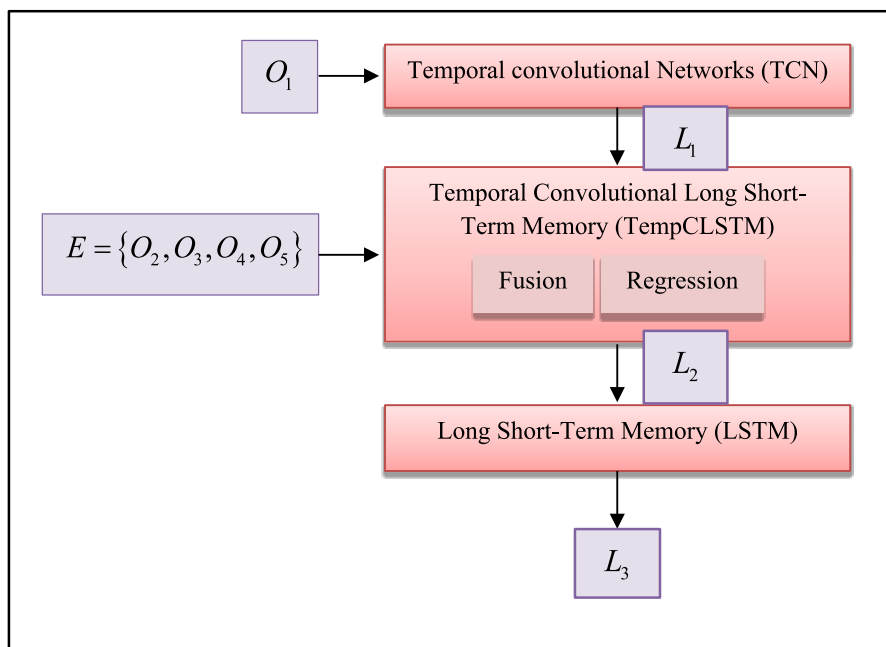


Fig. 4 Schematic outline of the TempCLSTM model

is an activation function that is employed after convolutional layers. These functions present non-linearity to the model that assists in learning complex temporal patterns. When the temporal convolutions have extracted the essential features from the sequence, then the output is typically passed through one or more Fully Connected Layers (FCL). FCL aggregates the learned features from all time steps and forwards them to the output layer to detect the TCN output L_1 as,

$$L_1 = \text{softmax}(T * O_1 + F) \quad (7)$$

where the weight matrix is explicated as T , the entropy-based one-hot encoding matrix is specified as O_1 , and bias is designated as F . The framework of TCN is represented in Fig. 5.

TempCLSTM: The mined features E and the TCN output L_1 are fed as input to the TempCLSTM layer. These inputs E and L_1 are combined using the regression model called Taylor series (Mangai et al. 2014) to generate the result of TempCLSTM layer as L_2 and is expressed as,

$$L_2 = 2 * \sum_{r=1}^u \sum_{l=1}^n E_{r,l} * G_{r,l} + L_1 \quad (8)$$

where the excerpted one-hot encoders are represented as $E_{r,l}$, and the weight is indicated as $G_{r,l}$, and the dimension of the one-hot encoder is explicated as u and n .

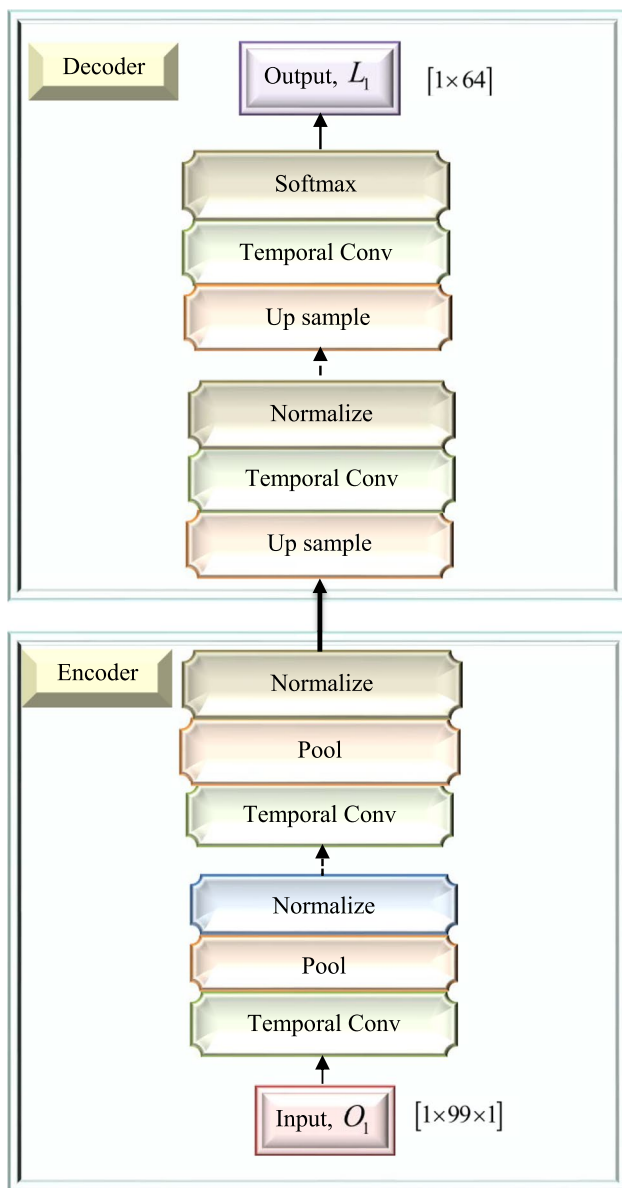


Fig. 5 Framework of TCN

Besides, L_1 value from Eq. (7) is substituted in Eq. (8) to attain the final result of the TempCLSTM layer.

$$L_2 = 2 * \sum_{r=1}^u \sum_{l=1}^n E_{r,l} * G_{r,l} + softmax(T * O_1 + F) \quad (9)$$

LSTM: The output from the TempCLSTM layer (L_2) is fed as an input to the LSTM. LSTM (Sagheer and Kotb 2019) is a specialized form of Recurrent Neural Network (RNN) that is designed by maintaining a memory cell that captures long-term dependencies and avoids the vanishing gradient issue. LSTM possesses a memory cell that runs through the system. This memory cell permits the system to recollect the information for long periods. LSTM comprises different gates that regulate the information flow. These gates regulate the process of updating the memory cell at each time step. The key gates in LSTM are forget gate, input gate, candidate cell state, update cell state, output gate, as well as hidden state. The forget gate regulates which information to remove from the cell state, the input gate decides which new information should be stored in the cell state, the candidate cell state designs the new value to add in the particular cell state, the update cell state ingrates the old cell state with new information for updating cell state, output gate determines which section of the memory cell will be the output, and finally, the output generated from the cell state is called as hidden state. The final output of LSTM L_3 is stated as,

$$L_3 = \wp((L_2) Z^x + W_{q-1} V^x + y_x) \otimes \tanh(w_\Omega) \quad (10)$$

where, sigmoid activation function is specified as $\wp$, the weight matrix linked with the input is specified as Z^x , the hidden state from the prior time step $q - 1$ is represented as W_{q-1} , the weight matrix linked with the previous hidden state W_{q-1} is specified as V^x , the bias term is signified as y_x , and the cell state at time step Ω is implied as w_Ω .

Substituting the value of L_2 from Eq. (9) in Eq. (10), we get,

$$L_3 = \wp\left(\left(2 * \sum_{r=1}^u \sum_{l=1}^n E_{r,l} * G_{r,l} + softmax(T * O_1 + F)\right) Z^x + W_{q-1} V^x + y_x\right) \otimes \tanh(w_\Omega) \quad (11)$$

Equation (11) represents the final output of the TempCLSTM model for detecting SQLi attacks. The framework of LSTM is represented in Fig. 6.

(ii) Proposed giant magnificent frigatebird optimization

The TempCLSTM model is trained using the devised Gi_MFO, which is the integration of MFO (Hamadneh et al. 2024a) and GTO (Sadeeq and Abdulazeez 2022). The MFO algorithm is inspired by the characteristics of a magnificent frigatebird, which is a seabird known for its unique feeding policy. The key characteristic of the MFO algorithm is based on the kleptoparasitic behavior of the magnificent frigatebird. Here, the bird targets other seabirds, often attacking them to force them to give up their prey. Once the prey is surrendered, the frigatebird quickly swoops down and grabs the food before it hits the water's surface. The MFO algorithm balances exploration and exploitation very effectively, which helps to avoid local optima and inspires finding the global optimum, which is crucial for complex optimization tasks. However, MFOs suffer from premature convergence and are sensitive to the population size. So, GTO (Sadeeq and Abdulazeez 2022) is combined with MFO to enhance the entire performance of the system. GTO is considered the large marine predator

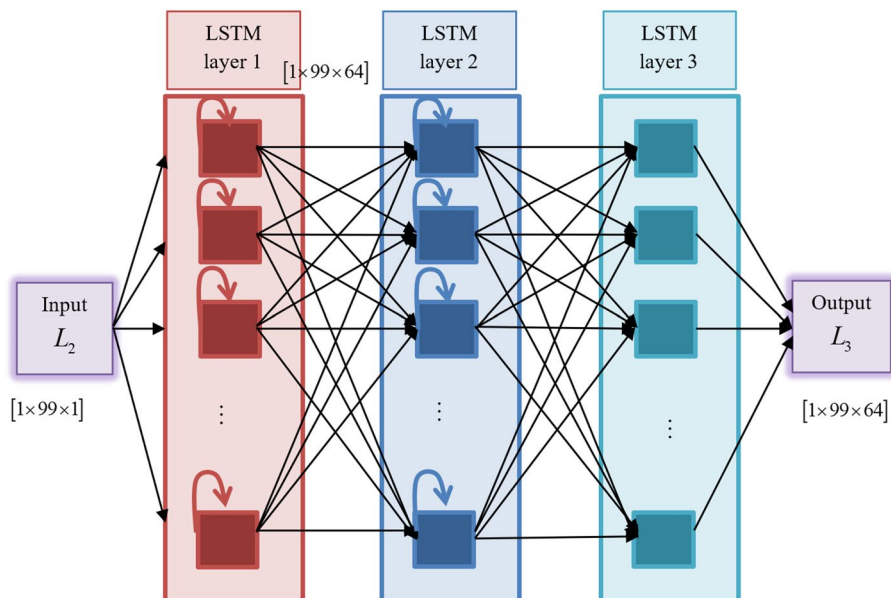


Fig. 6 Framework of LSTM

that is typically silver with dark spots and is easily recognizable by its sharp head, strong tail, and various other anatomical features. GTO dynamically adjusts its search behavior, and well-suited for multi-objective optimization tasks, and is strong in convergence rate. The mathematical model of Gi-MFO is detailed below,

Initialization: In Gi-MFO, each magnificent frigatebird signifies a candidate solution to the optimization problem. The position of each bird in search space is modelled as a vector, which corresponds to a potential solution for the specified optimization task. The position of each frigatebird is updated iteratively during the optimization process. The position of each bird is signified in the problem-solving space as a vector, and the position of the bird is adjusted in accordance with certain updated rules that are inspired by the bird's natural behavior, like hunting, foraging, and flying in specific patterns. The population of magnificent frigatebirds (H) represents the candidate solutions stored in the population matrix (I) with H rows and i columns.

$$I = \begin{bmatrix} I_1 \\ \vdots \\ I_2 \\ \vdots \\ I_H \end{bmatrix}_{H \times i} = \begin{bmatrix} o_{1 \times 1} & \cdots & o_{1 \times k} & \cdots & o_{1 \times i} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ o_{h \times 1} & \cdots & o_{h \times k} & \cdots & o_{h \times i} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ o_{H \times 1} & \cdots & o_{H \times k} & \cdots & o_{H \times i} \end{bmatrix}_{H \times i} \quad (12)$$

where $o_{h,k}$ denotes the k^{th} variable of the bird k .

The initial position of each bird in solution space is determined randomly within specified bounds for each decision variable. Each position of the bird is signified by a

vector of i values, where each value is randomly initialized between the lower bound (ℓ_k) and upper bound ($\mathcal{U}_k$) for that specific decision variable, as expressed as,

$$I : o_{h,k} = \ell_k + \tilde{h} \cdot (\mathcal{U}_k - \ell_k), \quad h = 1, 2, \dots, H, \quad k = 1, 2, \dots, i \quad (13)$$

where the random number drawn from the interval $[0,1]$ is specified as $\tilde{h}$.

Fitness function: MSE is evaluated as the average of squared differences between predicted outcomes and expected outcomes. The main objective during the optimization process is to minimize the MSE for finding the best solution. Fitness function (J) is expressed as,

$$J = \frac{1}{S} \sum_{z=1}^S (L_{3z}^* - L_3)^2 \quad (14)$$

where the predicted result of TempCLSTM is indicated as L_3 , the expected result is denoted L_3^* , and the entire number of data points in the dataset is represented as S .

Exploration stage: Each frigatebird deliberates the position of other members, which attained better fitness function values as targets that are similar to a predator (frigatebird), considering prey carried by other birds. The positions of these targeted birds for each frigate bird are,

$$M_h = \{I_\kappa, J_\kappa < J_h \text{ and } \kappa \in \{1, 2, \dots, H\}\}, \quad \text{where } h = 1, 2, \dots, H \quad (15)$$

where M_h denotes the set of potential food-carrying seabirds that are considered as targets by the h^{th} magnificent frigatebird, and I_κ indicates the κ^{th} row of the matrix I .

The algorithm assigns that a frigatebird chooses one of the targeted birds randomly, and then moves toward it, which is evaluated as,

$$o_{h,k}(m+1) = o_{h,k}(m) + (1 - 2\tilde{h}_{h,k}) \cdot (AA_{h,k} - K \cdot o_{h,k}(m)) \quad (16)$$

$$o_{h,k}(m+1) = o_{h,k}(m) + (1 - 2\tilde{h}_{h,k}) \cdot AA_{h,k} - (1 - 2\tilde{h}_{h,k}) \cdot K \cdot o_{h,k}(m) \quad (17)$$

$$o_{h,k}(m+1) = o_{h,k}(m) [1 - (1 - 2\tilde{h}_{h,k}) K] + (1 - 2\tilde{h}_{h,k}) \cdot AA_{h,k} \quad (18)$$

$$o_{h,k}(m+1) = o_{h,k}(m) [1 - K + 2\tilde{h}_{h,k} K] + (1 - 2\tilde{h}_{h,k}) \cdot AA_{h,k} \quad (19)$$

In addition, GTO (Sadeeq and Abdulazeez 2022) is applied to improve the performance of the system. GTO selects the region step that is inspired by the hunting characteristics of Giant Trevally fish. These fish find and select the best regions for hunting based on the abundance of food in the selected search space. This behavior is modelled mathematically in the algorithm to optimize the search for the best solutions as follows,

$$o_{h,k}(m+1) = Xp' \times B \times N + \mathfrak{S}_{\text{inf}} - o_{h,k}(m) \times N \quad (20)$$

where the present search space selected by the giant trevally, depending optimal position determined using the last search, is indicated as Xp' , position change controlling parameter is represented as B , random number is specified as N , and the mean, which denotes that the Giant trevally utilized all the accessible information from the preceding points, is designated as $\mathfrak{S}_{\text{inf}}$ and is evaluated as,

$$\mathfrak{S}_{\text{inf}} = \frac{1}{H} \sum_h^H o_{h,k}(m) \quad (21)$$

where the population count is explicated as H .

Equation (20) is modified as,

$$o_{h,k}(m) = \frac{(Xp' \times B \times N + \mathfrak{S}_{\text{inf}} - o_{h,k}(m+1))}{N} \quad (22)$$

Substitute the value of $o_{h,k}(m)$ from Eq. (22) in Eq. (19),

$$o_{h,k}(m+1) = \left(\frac{(Xp' \times B \times N + \mathfrak{S}_{\text{inf}} - o_{h,k}(m+1))}{N} \right) [1 - K + 2\hbar_{h,k}K] + (1 - 2\hbar_{h,k}) \cdot AA_{h,k} \quad (23)$$

$$\frac{o_{h,k}(m+1) * N + o_{h,k}(m+1) [1 - K + 2\hbar_{h,k}K]}{N} = \frac{\left(\frac{((1 - 2\hbar_{h,k}) \cdot AA_{h,k}) N}{+ (Xp' \times B \times N + \mathfrak{S}_{\text{inf}}) [1 - K + 2\hbar_{h,k}K]} \right)}{N} \quad (24)$$

$$o_{h,k}(m+1) [N + 1 - 2\hbar_{h,k}K + K] = \left(\frac{((1 - 2\hbar_{h,k}) \cdot AA_{h,k}) N}{+ (Xp' \times B \times N + \mathfrak{S}_{\text{inf}}) [1 - K + 2\hbar_{h,k}K]} \right) \quad (25)$$

$$o_{h,k}(m+1) = \frac{[(1 - 2\hbar_{h,k}) \cdot AA_{h,k} * N + (Xp' \times B \times N + \mathfrak{S}_{\text{inf}}) [1 - K + 2\hbar_{h,k}K]]}{[N + 1 - 2\hbar_{h,k}K + K]} \quad (26)$$

A new position is evaluated based on the first stage of Gi-MFO, and then the value of the fitness function of the new position is compared with the fitness function value of the present position. If the new position $J_h(m+1) \leq J_h$ is better or equal to the fitness function, then the new position replaces the present position; otherwise present position remains unchanged. The fitness is reevaluated as,

$$I_h = \begin{cases} I_h(m+1), & J_h(m+1) \leq J_h \\ I_h, & \text{else} \end{cases} \quad (27)$$

Exploitation stage: In the second phase of kleptoparasitic characteristics, the magnificent frigatebird dives to the prey unrestricted by the attacking seabird, and, catching it earlier, it attains the surface of the water. This characteristic is simulated in the algorithm through precise movements near the present position of the bird is $o_{h,k}(m+1)$,

$$o_{h,k}(m+1) = o_{h,k}(m) + (1 - 2h_{h,k}) \cdot \frac{X_k - o_{h,k}(m)}{m} \quad (28)$$

where the best member at dimension k is denoted as X_k , the iteration counter is represented as m , and random numbers from the interval $[0, 1]$ is indicated as $h_{h,k}$.

If the new position yields a better value of the objective function, then the bird updates its position and, potentially, its personal best with respect to the update rule.

$$I_h = \begin{cases} I_h^{m+1}, & J_h^{m+1} \leq J_h \\ I_h, & \text{else} \end{cases} \quad (29)$$

where the new position evaluated for magnificent frigatebird h depending on the second stage, is represented as I_h^{m+1} , and the fitness function value is indicated as J_h^{m+1} .

Repetition Process: MFO algorithm updates the positions of all magnificent frigatebirds after completing the first iteration through exploration and exploitation phases. After updation, this algorithm proceeds to the subsequent iteration with recently attained values. This reiterative process endures, following the guidelines set forth from Eqs. (15)–(29) till it reaches the maximum number of iterations.

Termination: The termination phase occurs when the algorithm attains a maximum number of iterations. Once the termination criteria are satisfied, the algorithm exits, and the greatest solution found so far is returned as the optimal solution.

Algorithm 1 exemplifies the pseudocode of the developed Gi-MFO.

Start Gi-MFO	
1	Input problem details.
2	Signify the count of iteration ($\tilde{\lambda}$) and magnificent frigatebird count (H)
3	Create the initial population randomly using equation (13)
4	Measure initial population
5	For $m = 1 : \tilde{\lambda}$
6	For $h = 1 : H$
7	Stage 1: Exploration
8	Find candidate food-carrying seabirds for frigatebird h by equation (15)
9	Update new position of frigatebird h by equation (26).
10	Update position of frigatebird h by equation (27)
11	Stage 1: Exploitation
12	Compute a new developed position for magnificent frigatebird h by equation (28)
13	Update the position of magnificent frigatebird h using equation (29).
14	end
15	Save the obtained best solution
16	end
17	Output: Attained optimal solution
End Gi-MFO	

Fig. 7 Pseudocode of the developed Gi-MFO

The final output gained is a more accurate and efficient classification of SQL queries as either normal or SQLi attacks. Gi-MFO helps by optimizing the parameters of TempCLSTM, improving accuracy, reducing training time, and increasing the ability of the system to generalize, leading to better detection of SQLi attacks in web applications.

4.2 Global model

Once the local models have been trained, their weights are sent to a central server. The server aggregates these local models to create a global model. After aggregation, the server creates a global model based on the averaged weights from the local models. The global model essentially summarizes the data from all the local models that were trained on varied datasets. Once the server aggregates the local models into a global model, it is distributed back to the local nodes for additional training. This iterative cycle of local training and global aggregation enables the model to improve continuously and enhance its ability to generalize across various datasets.

5 Results and discussion

The validation of TempCLSTM_Gi-MFO is particularly designed for detecting SQLi attacks. It is executed with various evaluation measures and compared with conventional approaches for establishing its efficiency.

5.1 Experimental setup

Python tool is utilized for SQLi attack detection to validate the efficiency of the TempCLSTM_Gi-MFO method. The simulation parameters of TCN, LSTM, and Gi-MFO are deliberated in Tables 3, 4, and 5.

5.2 Database description

The SQL injection dataset (The SQL Injection Dataset) is utilized for detecting SQLi attacks. The total sample size is 30,919. The label column in the dataset comprises raw SQL query strings as well as integer values of zero or one. In the Label column, zero denotes a non-malicious query and one specifies a malicious query. The Label column comprises integer values (0 or 1) that correspond to whether the SQL query in the associated row is malicious or not. Generally, the SQL injection dataset encompasses a column with the actual SQL query strings, which are the inputs that are processed and classified as either malicious (1) or non-malicious (0).

Table 3 Simulation parameters of TCN

Epoch	80
batch_size	64
activation	“sigmoid”
loss	‘binary_crossentropy’
Dropout	0.2
optimizer	Gi_MFO
kernerl_size	3
nb_filters	64
kernel_initializer	‘glorot_uniform’

Table 4 Simulation parameters of LSTM

Epoch	80
batch_size	64
activation	“sigmoid”
loss	‘binary_crossentropy’
Dropout	0.2
optimizer	Gi_MFO
kernel_regularizer	L2
kernel_initializer	‘glorot_uniform’

Table 5 Simulation parameters of Gi-MFOs

H (Population size)	50
λ (Maximum Iteration)	100
$\bar{h}$ (random number)	[0, 1]
N (random number)	1 or 2
ℓ_k (lower bound)	1
$\bar{U}_k$ (upper bound)	1
i (decision variable)	5

5.3 Performance parameters

Several evaluation measures are exploited to measure the performance of Temp-CLSTM_Gi-MFO for SQLi attack detection, which are detailed below,

- (i) *Accuracy*: Accuracy (Q') measures the entire performance of the TempCLSTM_Gi-MFO model. It is the ratio of correctly predicted instances, such as normal SQL and hacker SQL, to the total instances.

$$Q' = \frac{\Theta_1 + \Theta_2}{\Theta_1 + \Theta_2 + \Theta_3 + \Theta_4} \quad (30)$$

where Θ_1 specifies the number of SQLi attacks correctly classified as attacks, Θ_2 represents the normal queries correctly classified as normal, Θ_3 indicates the

normal queries incorrectly classified as SQLi attacks, and Θ_4 denotes the SQLi attacks incorrectly classified as normal queries.

- (ii) *Precision*: The proportion of positive instances (correctly identified SQLi attacks) among all instances that the model predicted as attacks (both true positives and false positives) is measured by precision. A high precision designates fewer false positives. Precision (U) is stated as,

$$U = \frac{\Theta_1}{\Theta_1 + \Theta_3} \quad (31)$$

- (iii) *Recall*: The proportion of actual SQLi attacks that were correctly identified by the model is defined as recall (R). A high recall indicates fewer false negatives and is represented as,

$$R = \frac{\Theta_1}{\Theta_1 + \Theta_4} \quad (32)$$

- (iv) *F1-Score*: The harmonic mean of precision and recall is known as the F1-score. A high F1-score (η) denotes a good balance between precision and recall and is represented as,

$$\eta = 2 \times \frac{U \times R}{U + R} \quad (33)$$

5.4 Data visualization

Figure 8 displays the data visualization graph of the TempCLSTM_Gi-MFO method using the SQL Injection Dataset. The deviation graph represents the deviation and sample count. Here, a normal SQL query is indicated as zero, and an attack SQL query is denoted as one. Deviation is referred to as the difference between a given data point and a reference value. It is used to measure how much a data point deviates from the reference point. If the actual value is zero and the predicted value is one, then the deviation value is represented as -1 , which indicates that the model wrongly detected an attack when it was normal. Similarly, if the actual value is one and the predicted value is one, then the deviation value is equal to zero, which indicates a correct prediction (either normal or attack was correctly predicted). Likewise, if the actual value is one and the predicted value is zero, then the deviation is equal to one, which represents a false negative.

5.5 Experimental outcome

The experimental results of the TempCLSTM_Gi-MFO method for both Normal SQL Query and Hacker SQL Query are explicated below.

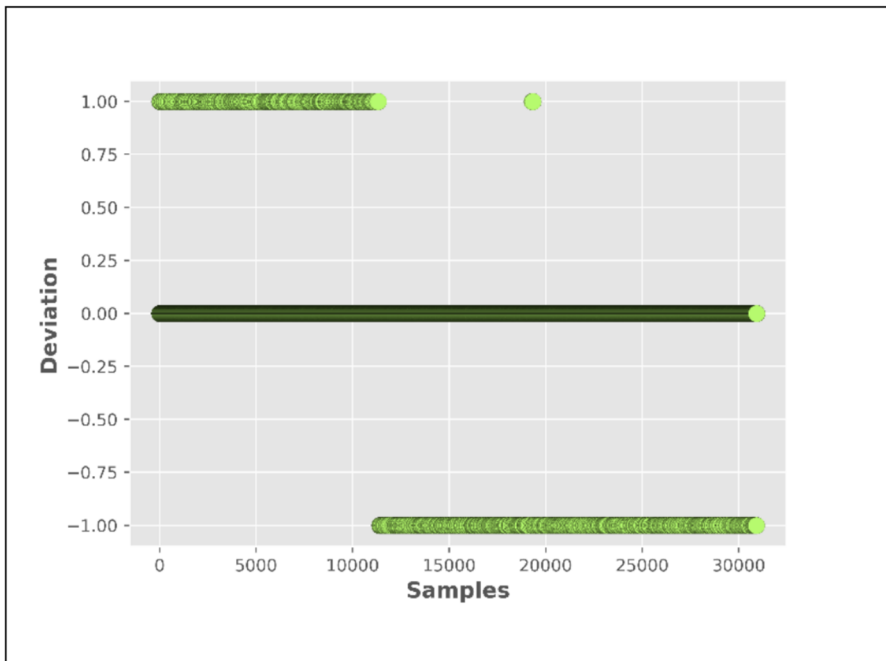


Fig. 8 Data Visualization of TempCLSTM_Gi-MFO

5.5.1 Normal SQL query

A normal SQL query is a legitimate query used to interact with a database for retrieving, inserting, updating, or deleting data. The input SQL Query, Tokenized Query, assigning an index to unique tokens, and one-hot vector for the normal SQL query are detailed below.

Input SQL Query: SELECT * FROM rocket FETCH FIRST 50 PERCENT ROWS ONLY SELECT TOP 3 * FROM ate.

Tokenized Query: “SELECT”, “*”, “FROM”, “rocket”, “FETCH”, “FIRST”, “50”, “PERCENT”, “ROWS”, “ONLY”, “SELECT”, “TOP”, “3”, “*”, “FROM”, “ate”.

Assign index to unique tokens: The index value is assigned for each unique token for normal SQL queries that are depicted in Table 6.

One hot vector: A one-hot vector is a binary vector representation where each unique token is signified by a vector with a value set as 1, and all other values are set to 0. Table 7 illustrates the one-hot vector for a normal SQL query.

5.5.2 Hacker SQL query

A Hacker SQL query refers to malicious SQL queries, which exploit vulnerabilities in the database to execute unauthorized commands. These kinds of queries are used to manipulate, access, or damage databases. The input SQL Query, Tokenized Query, assigned index to unique tokens, and one-hot vector for hacker SQL query are described below.

Input SQL Query: create user name identified by pass123 temporary tablespace temp default tablespace users.

Tokenized Query: “create”, “user”, “name”, “identified”, “by”, “pass123”, “temporary”, “tablespace”, “temp”, “default”, “tablespace”, “users”.

Assign index to unique tokens: The index value is allotted for each unique token for the hacker SQL query that is depicted in Table 8.

One hot vector: A one-hot vector is a binary vector representation where each unique token is signified by a vector with a value set as 1, and all other values are set to 0. Table 9 illustrates the one-hot vector for hacker SQL queries.

5.6 Performance validation

The assessment of TempCLSTM_Gi-MFO for detecting SQLi attacks is attained by varying the epochs with time steps and is represented in Fig. 9. The accuracy-based evaluation of TempCLSTM_Gi-MFO is illustrated in Fig. 9(i). The TempCLSTM_Gi-MFO achieved an accuracy of 93.987%, 92.184%, 91.000%, and 89.187% for 80, 60, 40, and 20 epochs with a time step of 25 s. Figure 9(ii) explicates the precision-based validation of TempCLSTM_Gi-MFO. At time step 25 s, TempCLSTM_Gi-MFO attained a precision of 89.998% for 40 epochs, 88.010% for 20 epochs, 92.795% for 80 epochs, and 91.003% for 60 epochs. The assessment of TempCLSTM_Gi-MFO concerning recall is depicted in Fig. 9(iii). TempCLSTM_Gi-MFO attained the recall of 91.009%, 89.998%, 94.478%, and 92.999% for 40, 20, 80, and 60 epochs at time step 25 s. The validation of TempCLSTM_Gi-MFO using F1-score is exemplified in Fig. 9(iv). TempCLSTM_Gi-MFO obtained an F1-score of 90.501%, 88.993%, 93.629%, and 91.990% for the number of epochs as 40, 20, 80, and 60, with a time step of 25 s.

Table 6 Token indexing for normal SQL query

Token	Index
SELECT	0
*	1
FROM	2
rocket	3
FETCH	4
FIRST	5
50	6
PERCENT	7
ROWS	8
ONLY	9
SELECT	0
TOP	10
3	11
*	1
FROM	2
ate	12

Table 7 One-hot vector for normal SQL query

Token	0	1	2	3	4	5	6	7	8	9	10	11	12
SELECT	1	0	0	0	0	0	0	0	0	0	0	0	0
*	0	1	0	0	0	0	0	0	0	0	0	0	0
FROM	0	0	1	0	0	0	0	0	0	0	0	0	0
rocket	0	0	0	1	0	0	0	0	0	0	0	0	0
FETCH	0	0	0	0	1	0	0	0	0	0	0	0	0
FIRST	0	0	0	0	0	1	0	0	0	0	0	0	0
50	0	0	0	0	0	0	1	0	0	0	0	0	0
PERCENT	0	0	0	0	0	0	0	1	0	0	0	0	0
ROWS	0	0	0	0	0	0	0	0	1	0	0	0	0
ONLY	0	0	0	0	0	0	0	0	0	1	0	0	0
SELECT	1	0	0	0	0	0	0	0	0	0	0	0	0
TOP	0	0	0	0	0	0	0	0	0	0	0	1	0
3	0	0	0	0	0	0	0	0	0	0	0	0	1
*	0	1	0	0	0	0	0	0	0	0	0	0	0
FROM	0	0	1	0	0	0	0	0	0	0	0	0	0
ate	0	0	0	0	0	0	0	0	0	0	0	0	1

5.7 Comparative techniques

The devised TempCLSTM_Gi-MFO scheme is compared with preceding detection models, like SQLNN (Zhang et al. 2022), MVC-BiCNN (Kakisim 2024), PNN (Alarfaj and Khan 2023), AE-Net (Thalji et al. 2023), TempCLSTM, synBERT (Lu et al. 2023), and BERT-LSTM (Liu and Dai 2024) to evaluate the efficacy of TempCLSTM_Gi-MFO for SQLi attack detection.

5.8 Comparative evaluation

Figure 10 depicts the validation of TempCLSTM_Gi-MFO and other baselines for detecting SQLi attacks with a time step. The assessment of TempCLSTM_Gi-MFO with accuracy is depicted in Fig. 10(i). At a time step of 25 s, the accuracy attained using TempCLSTM_Gi-MFO is 93.987%, whereas prevailing models, like AE-Net, obtained 84.887%, SQLNN achieved 85.999%, PNN attained 87.182%, MVC-BiCNN obtained 89.998%, TempCLSTM acquired 91.847, BERT-LSTM obtained 92.479%, and synBERT recorded 92.879%. This shows that the accuracy of TempCLSTM_Gi-MFO is 4.244% greater than the MVC-BiCNN scheme. Figure 10(ii) depicts the evaluation of TempCLSTM_Gi-MFO based on precision. At time step 25 s, TempCLSTM_Gi-MFO acquired a precision of 92.795%, whereas existing models, like AE-Net, SQLNN, PNN, MVC-BiCNN, TempCLSTM, BERT-LSTM, and synBERT attained a precision of 83.039%, 84.570%, 86.346%, 88.813%, 90.889%, 91.469%, and 91.878%. Here, the performance improvement of TempCLSTM_Gi-MFO is 8.863% when compared to SQLNN. The estimation of TempCLSTM_Gi-MFO using recall is delineated in Fig. 10(iii). The TempCLSTM_Gi-MFO reached a high recall of 94.478% for time step 25 s, and recall attained by the baseline schemes like AE-Net is 83.735%, SQLNN is 85.249%, PNN is 87.990%, MVC-BiCNN is 90.547%, TempCLSTM is 92.485%, BERT-LSTM is 93.155%, and synBERT is 93.479%. This infers that the recall of TempCLSTM_Gi-MFO is 6.868% higher than that of PNN. The assessment of TempCLSTM_Gi-MFO using F1-score is designated in Fig. 10(iv). For time step 25 s, the prior models, like AE-Net, SQLNN, PNN, MVC-BiCNN, TempCLSTM, BERT-LSTM, synBERT, and developed TempCLSTM_Gi-MFO acquired the F1-score of 83.385%, 84.908%, 87.160%, 89.672%, 91.680%, 92.304%, 92.671%, and 93.629%. Here, the F1-score of TempCLSTM_Gi-MFO is 10.940% better when compared with AE-Net.

5.9 Algorithm techniques

The designed Gi-MFO+TempCLSTM is compared with the existing schemes, like Gannet Optimization Algorithm (GOA) (Pan et al. 2022)+TempCLSTM, Fossa Optimization Algorithm (FOA) Hamadneh et al. (2024b)+TempCLSTM, GTO (Sadeeq

Table 8 Token indexing for hacker SQL query

Token	Index
create	0
user	1
name	2
identified	3
by	4
pass123	5
temporary	6
tablespace	7
temp	8
default	9
tablespace	7
users	10

and Abdulazeez (2022)+TempCLSTM, and MFO Hamadneh et al. (2024a)+to evaluate its performance in detecting SQLi attacks using the SQL Injection Dataset.

5.10 Algorithm assessment

The algorithm assessment of Gi-MFO+TempCLSTM with a time step is illustrated in Fig. 11. The analysis of Gi-MFO+TempCLSTM with accuracy based on the time step is represented in Fig. 11(i). For time step 25 s, the designed Gi-MFO+TempCLSTM attained the accuracy of 93.987%, and prevailing approaches, such as GOA+TempCLSTM, FOA+TempCLSTM, GTO+TempCLSTM, and MFO+TempCLSTM attained the accuracy of 86.990%, 88.009%, 89.949%, and 91.284%. This depicts that the accuracy of Gi-MFO+TempCLSTM is 7.445% superior to GOA+TempCLSTM. The validation of Gi-MFO+TempCLSTM using precision with time step is deliberated in Fig. 11(ii). The precision achieved by Gi-MFO+TempCLSTM is 92.795% at time step 25 s, whereas traditional methods, like GOA+TempCLSTM, FOA+TempCLSTM, GTO+TempCLSTM, and MFO+TempCLSTM attained the precision of 85.547%, 87.112%, 88.957%, and 90.181%. Here, the Gi-MFO+TempCLSTM recorded a high precision of 6.123% when compared with FOA+TempCLSTM. The assessment of GOA+TempCLSTM using recall with time step is represented in Fig. 11(iii). At time step 25 s, the recall attained by Gi-MFO+TempCLSTM is 94.478%, whereas the baseline schemes, like GOA+TempCLSTM, FOA+TempCLSTM, GTO+TempCLSTM, and MFO+TempCLSTM recorded the recall values of 86.987%, 88.966%, 90.050%, and 91.855%. Here, Gi-MFO+TempCLSTM is improved by 4.687% when compared with Gi-MFO+TempCLSTM. The examination of Gi-MFO+TempCLSTM by F1-score based on the time step is indicated in Fig. 11(iv). When time step is 25 s, then Gi-MFO+TempCLSTM attained the F1-score of 93.629%, whereas, prevailing schemes, namely GOA+TempCLSTM, FOA+TempCLSTM,

Table 9 One-hot vector for hacker SQL query

Token	0	1	2	3	4	5	6	7	8	9	10	11	12
create	1	0	0	0	0	0	0	0	0	0	0	0	0
user	0	1	0	0	0	0	0	0	0	0	0	0	0
name	0	0	1	0	0	0	0	0	0	0	0	0	0
identified	0	0	0	1	0	0	0	0	0	0	0	0	0
by	0	0	0	0	1	0	0	0	0	0	0	0	0
pass123	0	0	0	0	0	1	0	0	0	0	0	0	0
temporary	0	0	0	0	0	0	1	0	0	0	0	0	0
tablespace	0	0	0	0	0	0	0	1	0	0	0	0	0
temp	0	0	0	0	0	0	0	0	1	0	0	0	0
default	0	0	0	0	0	0	0	0	0	1	0	0	0
tablespace	0	0	0	0	0	0	0	1	0	0	0	0	0
users	0	0	0	0	0	0	0	0	0	0	0	0	1

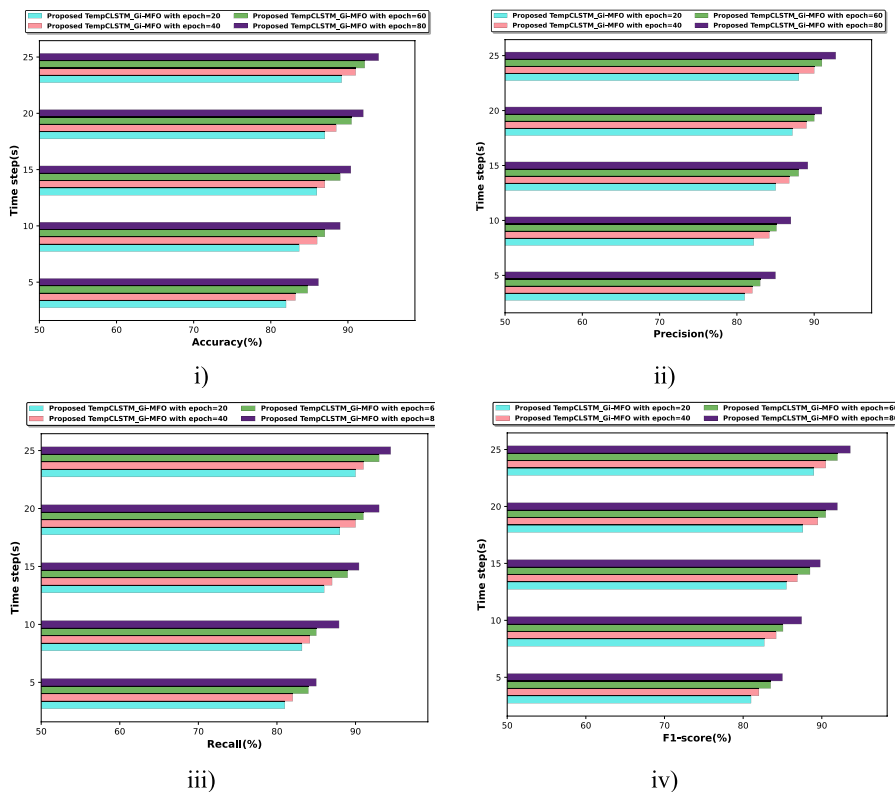


Fig. 9 Performance evaluation of TempCLSTM_Gi-MFO with time step by altering epochs—(i) Accuracy, (ii) Precision, (iii) Recall, (iv) F1-score

GTO+TempCLSTM, and MFO+TempCLSTM achieved the F1-score values of 86.261%, 88.029%, 89.500%, and 91.010%. This signifies that the F1-score of Gi-MFO+TempCLSTM is 2.797% higher than MFO+TempCLSTM.

5.11 Comparative discussion

Table 10 demonstrates the comparative discussion of TempCLSTM_Gi-MFO and prevailing models. Here, at time step 25 s, the accuracy attained by TempCLSTM_Gi-MFO is 93.987%, whereas the traditional systems, like AE-Net, SQLNN, PNN, MVC-BiCNN, TempCLSTM, BERT-LSTM, and synBERT acquired the accuracy of 84.887%, 85.999%, 87.182%, 89.998%, 91.847%, 92.479%, and 92.879%. Similarly, the precision-based validation of TempCLSTM_Gi-MFO is 90.889%, and existing techniques, like AE-Net acquired 83.039%, SQLNN achieved 84.570%, PNN obtained 86.346%, MVC-BiCNN achieved 88.813%, TempCLSTM attained 90.889%, BERT-LSTM squired 91.469%, and synBERT achieved 91.878%. Likewise, the recall attained by developed TempCLSTM_Gi-MFO, traditional AE-Net, SQLNN, PNN, MVC-BiCNN, TempCLSTM, BERT-LSTM, and synBERT is 83.735%, 85.249%, 87.990%, 90.547%, 92.485%,

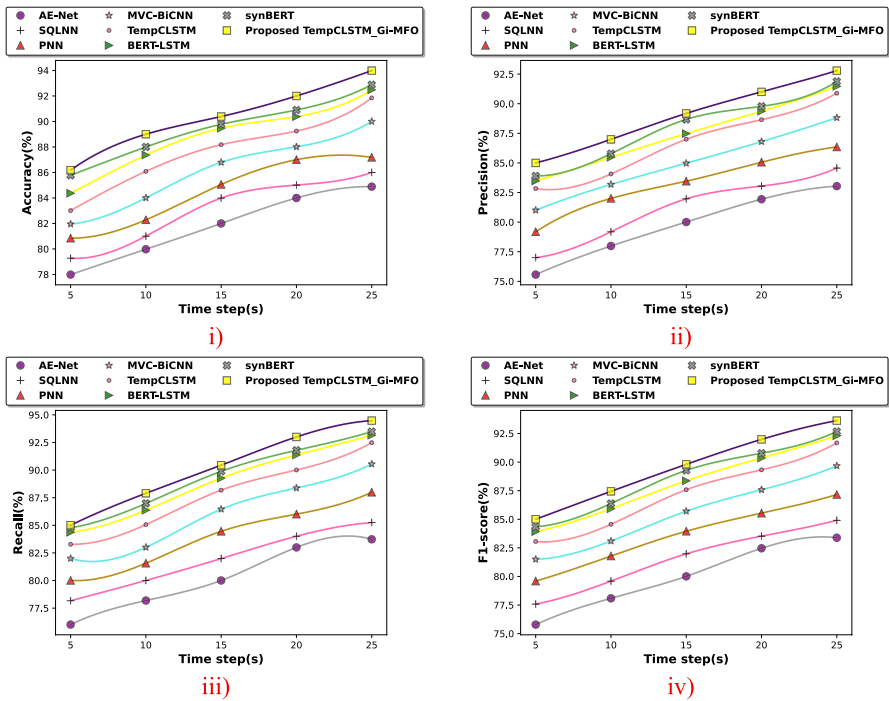


Fig. 10 Evaluation of TempCLSTM_Gi-MFO with time step—(i) Accuracy, (ii) Precision, (iii) Recall, (iv) F1-score

93.155%, and 93.479%. At last, the F1-score acquired by TempCLSTM_Gi-MFO is 91.680%, and traditional methods, like AE-Net, SQLNN, PNN, MVC-BiCNN, TempCLSTM, BERT-LSTM, and synBERT are 83.385%, 84.908%, 87.160%, 89.672%, 91.680%, 92.304%, and 92.671% at time step 25 s. TCN captures long-range dependencies in data more efficiently, and LSTM is effective for sequential data, as it possesses an explicit memory cell that allows for the storage of long-term context across time steps. TCN and LSTM are combined to introduce TempCLSTM for handling both short-term and long-term dependencies very effectively, and improving robustness. Furthermore, TempCLSTM is trained by the introduced Gi-MFO to enhance the performance of the whole model. Gi-MFO is the combination of GTO and MFO. MFO is effective in balancing both exploration and exploitation phases and avoids premature convergence, whereas GTO has a strong capability of local search.

5.12 Ablation study

An ablation study is a method used to evaluate the contribution of individual components of the devised model by removing or altering them. Figure 12 shows the ablation study of the proposed TempCLSTM_Gi-MFO model. While considering 25-time steps, the proposed TempCLSTM_Gi-MFO obtained the accuracy

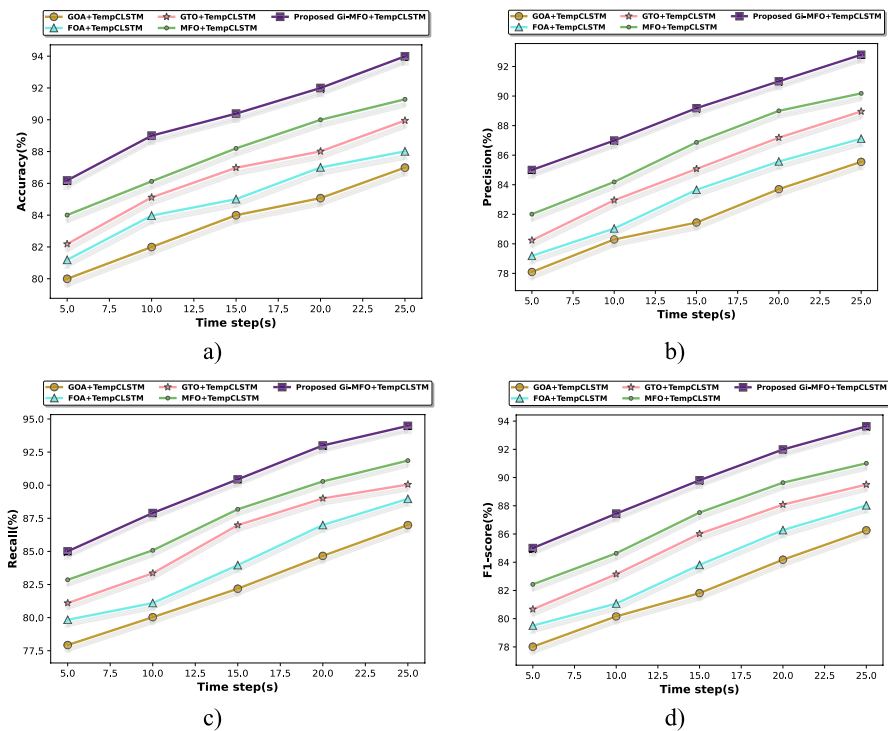


Fig. 11 Algorithm validation of Gi-MFO+TempCLSTM concerning time step—(i) Accuracy, (ii) Precision, (iii) Recall, (iv) F1-score

of 93.99%, while the other methods, like TempCLSTM_Gi-MFO with Entropy-Based One-Hot Encoding, TempCLSTM_Gi-MFO with Mutual Information-Based One-Hot Encoding, TempCLSTM_Gi-MFO with Information Gain-Based One-Hot Encoding, TempCLSTM_Gi-MFO with Correlation coefficient-Based One-Hot Encoding, and TempCLSTM_Gi-MFO with Gini index-Based One-Hot Encoding, obtained the accuracy of 88.00%, 88.99%, 90.00%, 91.00%, and 91.99%. The results demonstrate that each individual measure contributes uniquely to the model's performance, and their combined use significantly enhances detection accuracy.

5.13 Convergence analysis

Convergence analysis refers to the process of evaluating how and when the algorithm reaches a stable solution. Figure 13 gives the convergence analysis of the proposed Gi-MFO model. Here, the Gi-MFO algorithm is compared with the existing algorithms such as GOA, FOA, GTO, and MFO. At iteration 100, the devised Gi-MFO attained the fitness of $3.46\text{E-}06$, while the prevailing methods attained the fitness of $3.54\text{E-}03$, $1.31\text{E-}03$, $1.82\text{E-}04$, and $2.51\text{E-}05$. The minimum convergence of the devised model indicates that the optimizer has successfully found the best solution

Table 10 Comparative discussion

Evaluation measures (%)	Existing methods				Designed TempCLSTM_Gi-MFO		
	AE-Net	SQLNN	PNN	MVC-BiCNN	TempCLSTM	BERT-LSTM	synBERT
Time step 25 s							
Accuracy	84.887	85.999	87.182	89.998	91.847	92.479	92.879
Precision	83.039	84.570	86.346	88.813	90.889	91.469	91.878
Recall	83.735	85.249	87.990	90.547	92.485	93.155	93.479
F1-score	83.385	84.908	87.160	89.672	91.680	92.304	92.671

The bold values represent the best performance

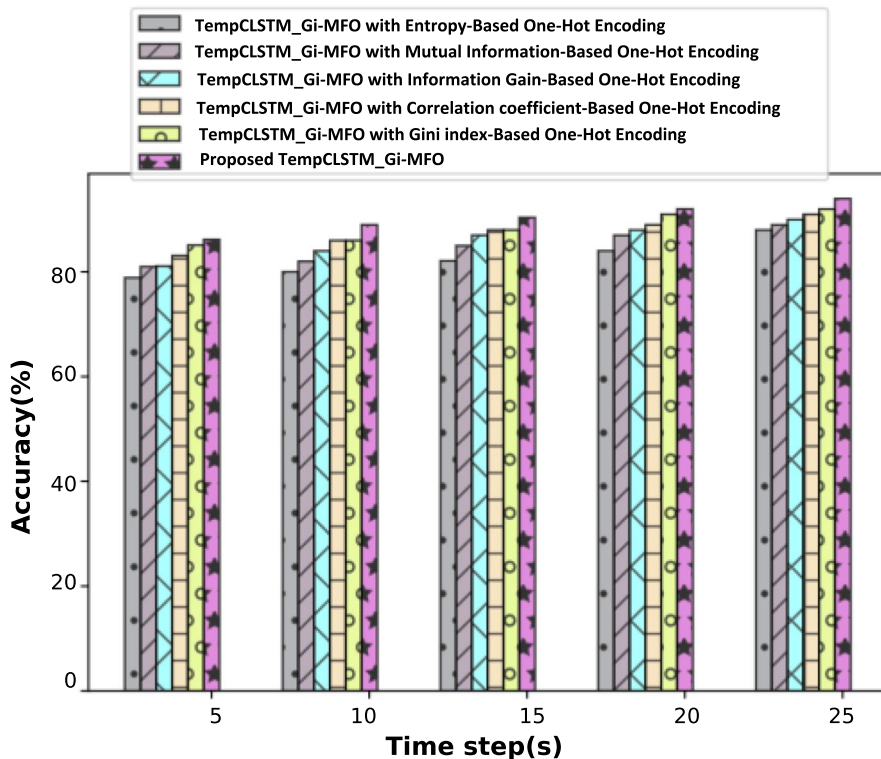


Fig. 12 Ablation study of one-hot encoding of the proposed TempCLSTM_Gi-MFO model

in a minimum time. The stability of the convergence curve shows the robustness and reliability of the devised model.

5.14 Analysis of computational complexity

The computational complexity is defined as the amount of time or resources required by the devised model as the size of the input increases.

Computational time refers to the amount of time required by a system to complete a task. Here, the devised TempCLSTM_Gi-MFO model attained the minimum computational time of 10.999 s.

Inference time refers to the amount of time taken by the method to produce a prediction after receiving the input. The proposed TempCLSTM_Gi-MFO model recorded the minimum inference latency of 3.998 s.

Memory refers to the amount of storage space required by the devised model during training and inference. The devised TempCLSTM_Gi-MFO required the minimum memory of 5.6 MB. Table 11 shows the computational complexity of the proposed TempCLSTM_Gi-MFO model.

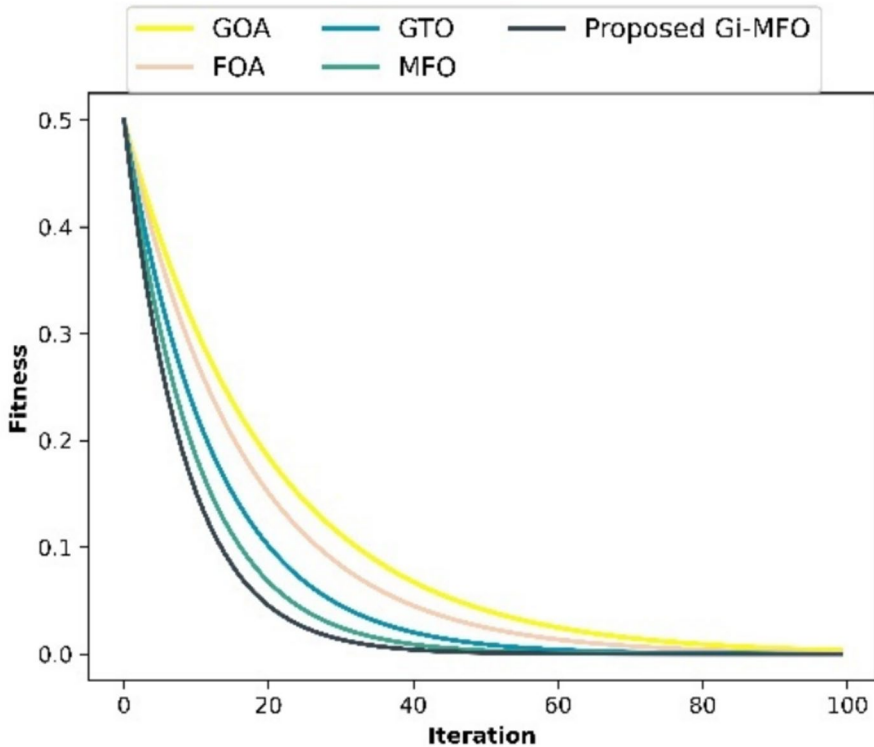


Fig. 13 Convergence analysis of the proposed Gi-MFO

Table 11 Analysis of the computational complexity of TempCLSTM_Gi-MFO

Metrics	AE-Net	SQLNN	PNN	MVC-BiCNN	Temp-CLSTM	BERT-LSTM	syn-BERT	Temp-CLSTM_Gi-MFO
Computational Time (sec)	35.988	29.988	25.999	20.999	17.999	15.998	13.010	10.999
Inference Latency (sec)	15.877	13.879	11.579	9.676	8.888	7.989	5.999	3.998
Memory (MB)	25.5	21.6	17.5	15.6	13.5	11.5	8.6	5.6

The bold values represent the best performance

6 Conclusion

SQLi attack detection is a critical aspect of maintaining the security and integrity of web applications. In this work, the integration of advanced techniques, such as FL, one-hot encoding methods, and optimized models significantly enhances the detection capabilities without compromising user privacy. FL is utilized for detecting SQLi attacks, enabling collaborative and decentralized model training across multiple devices with-

out sharing sensitive data. Initially, the submitted SQL queries are altered into binary sequences. Subsequently, one-hot encoding schemes, such as Mutual Information, Entropy, Information Gain, Correlation Coefficient, and Gini Index, are applied to convert the queries into one-hot vectors. The processed data is then fed to the SQLi attack detection system to determine whether the SQL query is normal or malicious using by TempCLSTM_Gi-MFO model. TempCLSTM is trained by Gi-MFO, which integrates both GTO and MFO. As a result, the TempCLSTM_Gi-MFO attained an accuracy of 91.847%, an F1-score of 91.680%, a recall of 92.485%, and a precision of 90.889%, with a time step of 25 s. The future directions for SQLi attack detection will utilize other DL techniques to focus on enhancing model performance and privacy, while adapting to emerging threats and ensuring transparency through explainable results.

Acknowledgements I would like to express my very great appreciation to the co-authors of this manuscript for their valuable and constructive suggestions during the planning and development of this research work.

Author contribution All authors have made substantial contributions to the conception and design, revising the manuscript, and the final approval of the version to be published. Also, all authors agreed to be accountable for all aspects of the work in ensuring that questions related to the accuracy or integrity of any part of the work are appropriately investigated and resolved.

Funding This research did not receive any specific funding.

Data availability The data underlying this article are available in the SQL Injection Dataset that is collected from "<https://www.kaggle.com/datasets/sajid576/sql-injection-dataset>".

Declarations

Conflict of interest The authors declare no conflict of interest.

Ethical approval Not applicable.

Informed consent Not applicable.

References

- Alarfaj FK, Khan NA (2023) Enhancing the performance of SQL injection attack detection through probabilistic neural networks. *Appl Sci* 13(7):4365
- Alghawazi M, Alghazzawi D, Alarifi S (2023) Deep learning architecture for detecting SQL injection attacks based on RNN autoencoder model. *Mathematics* 11(15):3286
- Arasteh B, Aghaei B, Farzad B, Arasteh K, Kiani F, Torkamanian-Afshar M (2024a) Detecting SQL injection attacks by binary gray wolf optimizer and machine learning algorithms. *Neural Comput Appl* 36(12):6771–6792
- Arasteh B, Bouyer A, Sefati S, Craciunescu R (2024b) Effective SQL injection detection: a fusion of binary olympiad optimizer and classification algorithm. *Mathematics* 12(18):2917
- Benesty J, Chen J, Huang Y (2008) On the importance of the Pearson correlation coefficient in noise reduction. *IEEE Trans Audio Speech Lang Process* 16(4):757–765
- Chen D, Yan Q, Wu C, Zhao J, Sql injection attack detection and prevention techniques using deep learning. In: *Journal of physics: conference series*, vol 1757(1). IOP publishing, pp 012055
- Crespo-Martínez L, Campazas-Vega A, Guerrero-Higueras A, Riego-DelCastillo V, Álvarez-Aparicio C, Fernández-Llamas C (2023) SQL injection attack detection in network flow data. *Comput Secur* 127:103093

- Deriba F, Salau A, Mohammed S, Kassa T, Demilie W (2022) Development of a compressive framework using machine learning approaches for SQL injection attacks. *Przegl Elektrotech* 7(1):181–187
- Elayaraja P, Suganthi M (2018) Automatic approach for cervical cancer detection and segmentation using neural network classifier. *Asian Pac J Cancer Prev APJCP* 19(12):3571
- Hamadneh T, Kaabneh K, AbuFalahah I, Bektemyssova G, Shaikemelev G, Umutkulov D, Omarov S, Monrazeri Z, Werner F, Dehghani M (2024a) Magnificent frigatebird optimization: a new bio-inspired metaheuristic approach for solving optimization problems. *Comput Mater Contin* 80(2):2721–2741
- Hamadneh T, Batiha B, Werner F, Montazeri Z, Dehghani M, Bektemyssova G, Eguchi K (2024b) Fossa optimization algorithm: a new bio-inspired metaheuristic algorithm for engineering applications. *Int J Intell Eng Syst* 17(5):1038–1047
- Hossain M, Islam M (2024) Enhancing DDoS attack detection with hybrid feature selection and ensemble based classifier: a promising solution for robust cybersecurity. *Meas Sens* 32:101037
- Jianping W, Guangqiu Q, Chunming W, Weiwei J, Jiahe J (2024) Federated learning for network attack detection using attention-based graph neural networks. *Sci Rep* 14(1):19088
- Kakisim AG (2024) A deep learning approach based on multi-view consensus for SQL injection detection. *Int J Inf Secur* 23(2):1541–1556
- Khan N, Abdullah J, Khan A (2017) A dynamic method of detecting malicious scripts using classifiers. *Adv Sci Lett* 23(6):5352–5355
- Lea C, Flynn M, Vidal R, Reiter A, Hager G (2017) Temporal convolutional networks for action segmentation and detection. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp 156–165
- Liu Y, Dai Y (2024) Deep learning in cybersecurity: a hybrid BERT–LSTM network for SQL injection attack detection. *IET Inf Secur* 2024:1–16
- Lu D, Fie J, Liu L (2023) A semantic learning-based SQL injection attack detection technology. *Electronics* 12(6):1344
- Mangai S, Sankar B, Alagarsamy K (2014) Taylor series prediction of time series data with error propagated by artificial neural network. *Int J Comput Appl* 89(1):2014
- Pan J, Zhang L, Wang R, Snášel V, Chu S (2022) Gannet optimization algorithm: a new metaheuristic algorithm for solving engineering optimization problems. *Math Comput Simul* 202:343–373
- Prasetyo B, Muslim M, Baroroh N (2021) Evaluation of feature selection using information gain and gain ratio on bank marketing classification using naïve bayes. In: *Journal of physics: conference series*, vol 1918(4), pp 042153
- Sadeeq H, Abdulazeez A (2022) Giant trevally optimizer (GTO): a novel metaheuristic algorithm for global optimization and challenging engineering problems. *IEEE Access* 10:121615–121640
- Sadiqâ A, Musawi K (2018) Modify random forest algorithm using hybrid feature selection method. *Int J Percept Cogn Comput* 4(2):1–6
- Sagheer A, Kotb M (2019) Time series forecasting of petroleum production using deep LSTM recurrent networks. *Neurocomput* 323(203):213
- Sefati S, Fartu O, Nor A, Halunga S (2024) Enhancing internet of things security and efficiency: anomaly detection via proof of stake blockchain techniques. In: *2024 International conference on artificial intelligence in information and communication (ICAIIIC)*, pp 591–595
- Sun H, Du Y, Li Q (2023) Deep learning-based detection technology for SQL injection research and implementation. *Appl Sci* 13(16):9466
- Thalji N, Raza A, Islam M, Samee N, Jamjoom M (2023) AE-net: novel autoencoder-based deep features for SQL injection attack detection. *IEEE Access* 11:135507–135516
- The SQL Injection Dataset. [Online]. Available: <https://www.kaggle.com/datasets/sajid576/sql-injection-dataset>. Accessed February 2025
- Zhang W, Li Y, Li X, Shao M, Mi Y, Zhang H, Zhi G (2022) Deep neural network-based SQL injection detection method. *Secur Commun Netw* 2022(1):4836289

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.